

FLUJO DE EJECUCIÓN: LA DIRECTIVA NEXT()

Arquitectura de Middlewares en Express.js - Unidad Final

1. Anatomía de la Directiva next()

A nivel de arquitectura de hardware y software, un servidor web no es más que una tubería de procesos (pipeline). Cuando una solicitud HTTP ingresa al puerto de tu servidor, Express la inyecta en una cadena de funciones llamadas **Middlewares**.

La función `next()` es el **puntero de control de flujo**. Es el interruptor que le dice al procesador: *"Terminé de ejecutar mi bloque de memoria, pasale los objetos request y response a la siguiente función en la fila"*.

La Regla de Oro del I/O (Input/Output):

El ciclo de vida de una petición HTTP **debe terminar siempre**. Tienes solo dos opciones matemáticas válidas en un middleware o controlador:

1. Enviar una respuesta al cliente (cortar el cable) usando `res.send()`, `res.json()`, etc.
2. Delegar el procesamiento al siguiente bloque usando `next()`.

Si no haces ninguna de las dos, el cliente (navegador/app) se queda colgado esperando una respuesta que nunca llegará, provocando un **Timeout** y saturando los sockets abiertos de tu servidor (Fuga de recursos).

2. Los 4 Vectores de Ejecución de next()

Dependiendo de los argumentos que le inyectes al invocarla, la directiva muta su comportamiento y redirige el flujo de datos por diferentes rutas de la memoria:

Vector de Invocación	Comportamiento en Hardware/Flujo
<code>next()</code>	Desplaza el puntero a la siguiente función exacta dentro del array de middlewares de la ruta actual.
<code>next('route')</code>	Aborta la cadena de middlewares actual y salta directamente al siguiente patrón de ruta que coincida en tu enrutador.
<code>next(error)</code>	Interrupción crítica. Salta absolutamente todos los middlewares estándar y aterriza en el Middleware de Manejo de Errores .
Ausencia de código	Falla arquitectónica. El hilo de ejecución se bloquea en el middleware actual. El cliente recibe un Timeout .

3. Arquitectura del "Firewall HTTP" (Middleware Condicional)

El uso más potente de `next()` es implementar cortafuegos de seguridad y validación. Antes de que tu base de datos reciba carga de I/O, el middleware intercepta el paquete de red y decide si el usuario tiene los privilegios adecuados.

```
// Implementación de un escudo de seguridad
const autenticar = (req, res, next) => {
  // Escaneo de cabeceras de red
  if (req.headers.authorization) {
    // Validación superada: El procesador delega al siguiente
    bloque
    next();
  } else {
    // Bloqueo de intrusión: Se aborta la cadena y se escupe el
    error 401
    res.status(401).send('Acceso denegado: Credenciales
    inexistentes');
  }
};

// Enganche del firewall en el endpoint
app.get('/admin/dashboard', autenticar, (req, res) => {
  res.send('Dashboard clasificado. Acceso concedido.');
```

En este bloque, si la condición falla, el `res.status(401)` actúa como un *return* terminal. El procesador descarta el resto de la ruta y libera el socket.

4. Ruteo de Excepciones: next(error)

Cuando trabajas con bases de datos o promesas asíncronas, los fallos de hardware (caída del disco, pérdida de conexión a PostgreSQL) van a ocurrir. En Express, la forma de evitar que la aplicación colapse (Crash) es atrapar el error y pasarlo por el canal seguro.

```
app.get('/datos-criticos', async (req, res, next) => {
  try {
    // Intento de extracción de datos en memoria
    const datos = await extraerDatosDelDisco();
```

```
        res.json(datos);
    } catch (error) {
        // Inyección de la excepción en el bus de errores
        next(error);
    }
});

// El Recolector de Basura / Gestor de Fallos (Debe estar al final de
app.js)
// Obligatoriamente recibe 4 parámetros para que el compilador lo
identifique
app.use((err, req, res, next) => {
    console.error('CRASH DETECTADO:', err.stack);
    res.status(500).send('Falla catastrófica en el clúster. Reintente
más tarde.');
```

5. Composición de Pipelines Multi-Fase

Un controlador monolítico es una aberración de software. La arquitectura moderna exige dividir las tareas en pequeñas funciones aisladas que pasan el testigo mediante `next()`. Esto te permite testear y reutilizar bloques individuales sin alterar el ecosistema.

```
// Endpoint de alta seguridad orquestado en 3 fases:
app.post('/transferencia',
  // Fase 1: Escudo de formato
  (req, res, next) => {
    if (!req.body.monto) return res.status(400).send('Payload corrupto');
    next(); // Puntero avanza a Fase 2
  },
  // Fase 2: Escudo de autenticación y carga de recursos
  async (req, res, next) => {
    const usuario = await BaseDeDatos.buscar(req.body.id);
    if (!usuario) return res.status(404).send('Usuario fantasma');

    // Almacenamos el dato en el objeto req para la siguiente fase
    req.usuarioValidado = usuario;
    next(); // Puntero avanza a Fase 3
  },
  // Fase 3: Controlador Transaccional
  async (req, res) => {
    // Consume el dato preparado en la Fase 2
    const recibo = await procesarPago(req.usuarioValidado, req.body.monto);

    // Fin del ciclo de vida HTTP. Socket liberado.
    res.json({ estado: 'Transacción exitosa', recibo });
  }
);
```

Fijate que en la Fase 2, mutamos el objeto `req` (`req.usuarioValidado = usuario`). Esta es la técnica estándar en hardware lógico para pasar datos en memoria RAM entre un middleware y otro sin golpear la base de datos dos veces.